

Software Maintenance & Re-Engineering

Comprehensive Study Notes • Practice Case Studies • Past Paper Solutions • Exam Traps



Core Theory & Concept Notes

1. Lehman's Laws of Software Evolution

Formulated by **Meir M. Lehman** and **Laszlo Belady** to describe the dynamics of evolving E-type (embedded in real-world environments) software systems:

- **The Law of Continuing Change:** A program used in a real-world environment must undergo continuous change, or it progressively becomes less useful in that environment.
- **The Law of Increasing Complexity:** As a program evolves, its complexity naturally increases unless extra resources are dedicated to preserving and simplifying its structure.



Core Note on Complexity: Every software modification inherently adds entropy to the architectural design. If refactoring and restructuring resources are not allocated alongside feature development, technical debt compounds exponentially.

2. The 17-18-65 Maintenance Effort Distribution

Empirical breakdown of effort across modern software maintenance activities:

Maintenance Category	Effort Percentage	Primary Scope & Description
Corrective	17%	Repairing runtime defects, logic errors, and unexpected bugs reported post-deployment.
Adaptive	18%	Porting the system to new environments (OS updates, database migrations, cloud hosting, API changes).
Perfective	65%	Adding new business features, improving user experience, enhancing performance, and structural refactoring.

3. Legacy Systems Management

A **legacy system** is a highly valuable, operational asset that an organization has inherited. It is reliable and runs large-scale business operations, but it suffers from outdated technology, high maintenance costs, and undocumented design drift.

When dealing with a legacy system, organizations have three strategic paths:

- **Leave Unchanged:** Continue standard operational maintenance. Appropriate when the component is stable, low-churn, and meets current operational demands (*"If it ain't broke, don't fix it"*).
- **Scrap and Replace:** Completely rewrite or purchase off-the-shelf software from scratch. Extremely high risk, costly, and frequently leads to schedule slips and lost domain logic.
- **Re-engineer:** Incrementally transform and modernize the system's internal structure and platform while retaining its validated business logic.

4. Software Re-Engineering

Definition: The examination and alteration of a subject system to reconstitute it in a new form, and the subsequent implementation of the new form. Importantly, **core functionality remains identical**; it is the internal structure, understandability, and maintainability that improve.

- **Primary Goal:** Reduce the complexity of a legacy system sufficiently so it can continue to be used and adapted at an acceptable total cost of ownership.
- **Key Drivers:** Unbundling monolithic architectures, porting to modern cloud/OS platforms, extracting undocumented business logic, improving performance, and reducing human knowledge silos.

5. The Horseshoe Lifecycle Model

The Horseshoe Model conceptualizes software re-engineering along three interconnected architectural dimensions:

- **Reverse Engineering (Bottom-Up):** Analyzing the monolithic legacy source code to extract higher-level representations (code structure → function-level abstraction → architectural models).
- **Transformation:** Restructuring and refactoring the high-level architecture, program plans, and design patterns from the old platform to the target platform requirements.
- **Forward Engineering (Top-Down):** Synthesizing the transformed high-level models down into clean, modular target source code on the new platform.

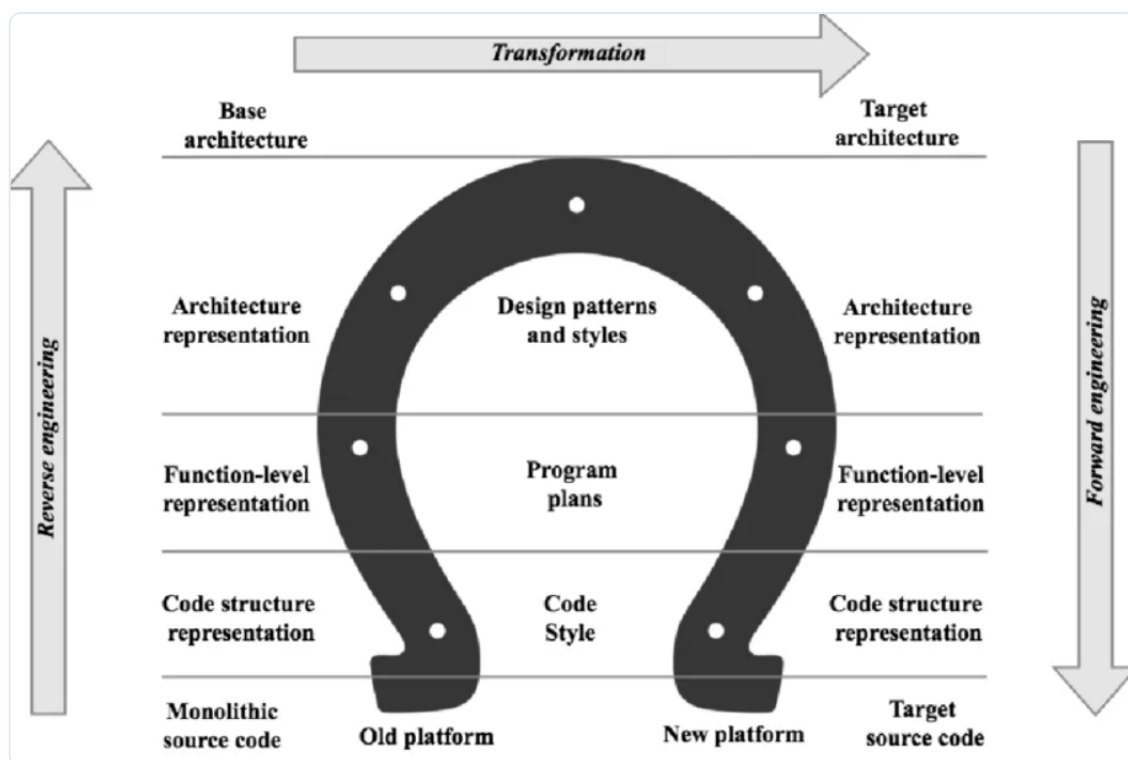


Figure: The Horseshoe Lifecycle Model for Software Re-Engineering

6. Restructuring vs. Refactoring

Dimension	Restructuring	Refactoring
Definition	Transforming from one representation to another at the same abstraction level while preserving behavior.	Behavior-preserving restructuring specifically within an Object-Oriented context.
Automation Level	Often fully automated (e.g., source-to-source compilers, AST translations).	Primarily manual developer insight supported by IDE automated refactoring tools.
Examples	Automated Pascal-to-C translation, reformatting control-flow structures.	Extract Method, Replace Conditional with Polymorphism, Rename Field.

7. Categories of Re-Engineering

- **Source Code Translation (Completely Automated):** Using parsing tools to translate code directly from one language to another at the same abstraction level without changing structure.
- **Reverse Engineering (Mostly Automated with Manual Intervention):** Analyzing software artifacts to identify system components and interrelationships.
- **Refactoring / Program Structure Improvement (Manual + IDE Support):** Modifying code internals to eliminate code smells and improve cohesion.
- **Program Modularization (Manual with Tool Support):** Clustering related functions/files into distinct decoupled modules or services.
- **Data Re-engineering (Tool-Driven):** Migrating legacy schemas (e.g., hierarchical or flat files) into modern relational/NoSQL schemas while cleaning corrupted data.

Practice Case Studies & Model Answers

Practice Question 1: Automated Language Translation Evaluation

Scenario: A company operating a legacy medical system decides to translate its entire codebase from Pascal to Java using a 100% automated translation tool. The CEO declares: *"Our system is now successfully re-engineered, and our maintenance costs will drop instantly."* Evaluate this statement using Lehman's Laws and the Re-Engineering Lifecycle.

Model Answer:

The CEO's declaration is **fundamentally flawed**:

1. **Restructuring vs. True Re-Engineering:** A 100% automated translation is merely *source-code translation (restructuring)*. It operates horizontally at the syntax level without ascending the Horseshoe Model to extract, understand, or redesign high-level architectural abstractions.
2. **Lehman's Law of Increasing Complexity:** Automated translation blindly converts procedural Pascal idioms into non-idiomatic, procedural-style Java ("Pascal written in Java syntax"). Because no resources were dedicated to improving design patterns or modularity, the overall structural complexity increases, making it harder for modern Java developers to maintain.
3. **Impact on Maintenance Costs:** Maintenance costs will actually *increase*. Unreadable translated code combined with lack of documentation creates severe design drift and technical debt, contradicting the goal of lowering maintenance overhead.

Practice Question 2: Monolithic Banking Modernization

Scenario: A banking firm owns a monolithic COBOL legacy system that has run flawlessly for 25 years. However, the business now wants to integrate real-time mobile banking. The original developers have all retired, and no documentation exists. The CIO is split between:

- **Option A:** Scrap and Replace (building a new cloud system from scratch).
- **Option B:** Re-engineer the legacy system.

Apply the OORP patterns *"Most Valuable First"* and *"If It Ain't Broke, Don't Fix It"* to advise the CIO.

Model Answer:

Recommendation: Advise the CIO against Option A (Scrap & Replace) and execute a targeted, incremental re-engineering strategy:

1. **Apply *"If It Ain't Broke, Don't Fix It"*:** The core COBOL transaction engine has executed flawlessly for 25 years and contains validated business rules. Rewriting it from scratch creates massive risk, high costs, and probable financial discrepancies. The core engine should remain untouched and encapsulated.
2. **Apply *"Most Valuable First"*:** Focus re-engineering efforts exclusively on the highest-value requirement: real-time mobile integration.
3. **Implementation Strategy:** Wrap the legacy COBOL system with modern REST/gRPC API integration layers (API Facade / Strangler Fig pattern). This exposes banking services to mobile apps safely without destabilizing the battle-tested backend logic.



Past Paper Questions & Detailed Solutions

Past Paper Q1: Software Maintenance vs. Software Re-Engineering

Question: Differentiate between software maintenance and software re-engineering in terms of objectives and activities involved.

Aspect	Software Maintenance	Software Re-Engineering
Primary Objectives	Keep the system operational and compliant in a changing environment through incremental bug fixes, platform adaptations, and requested feature enhancements.	Reconstitute the system in a new physical form, significantly reducing architectural complexity to restore long-term maintainability.
Activities Involved	Governed by the 17-18-65 rule : Corrective bug fixing (17%), Adaptive environment porting (18%), and Perfective feature addition/refactoring (65%).	Governed by the Horseshoe Lifecycle : Bottom-up reverse engineering, high-level architectural transformation, forward engineering, data restructuring, and modularization.

Past Paper Q2: Reverse Engineering vs. Forward Engineering

Question: Differentiate between Reverse Engineering and Forward Engineering.

- **Forward Engineering (Top-Down):** The traditional software development process moving from high-level, implementation-independent abstractions (requirements specifications, architectural designs) down to concrete, platform-specific source code and binary executables.
- **Reverse Engineering (Bottom-Up):** The process of analyzing an existing software system to identify its components and interrelationships, creating representations of the system at a higher abstraction level (e.g., extracting class diagrams, call graphs, or data models from raw source code).

Past Paper Q3: Top-Down vs. Bottom-Up Reverse Engineering

Question: Differentiate between Top-Down and Bottom-Up Reverse Engineering.

- **Top-Down Reverse Engineering:** Hypothesis-driven recovery. The engineer starts with high-level domain knowledge, business concepts, or mental models of system functionality and maps them downward to the underlying source code artifacts.
- **Bottom-Up Reverse Engineering:** Artifact-driven recovery. The engineer starts directly with the low-level source code, ASTs, and data structures, aggregating and abstracting them upward step-by-step into architectural modules, control-flow graphs, and design models.

Past Paper Q4: Technical Debt in Software Re-Engineering

Question: Explain Technical Debt in the context of Software Re-Engineering.

Definition & Impact: Technical debt represents the future implied cost of additional rework caused by choosing an easy, quick-and-dirty shortcut now instead of an architecturally sound solution. In legacy re-engineering, accumulated technical debt manifests as brittle code, tight coupling, and undocumented dependencies. Re-engineering acts as the principle debt-repayment mechanism, modernizing architecture and refactoring code to lower ongoing operational maintenance costs.

Examiner Tricks & Traps to Watch Out For

1. The "Maintenance Equals Bug Fixing" Trap:

The examiner will suggest that maintenance budgets can be slashed by outsourcing "bug-fixing" to junior programmers.

Counter-Argument: Cite the **17-18-65 rule**. Corrective bug fixes represent only 17% of total maintenance effort. Junior developers cannot independently handle perfective additions (65%) or environmental adaptations (18%), both of which demand deep architectural knowledge and domain expertise.

2. The "Clean Binary Reverse Engineering" Trick:

The examiner might ask you to reverse engineer a compiled binary (.exe or raw assembly) to rebuild the original complete software design.

Counter-Argument: Compiled binaries lose high-level semantics, variable names, type metadata, and design contracts during compilation. Decompilation is merely *restructuring / translation*. True re-engineering requires higher-level architectural design recovery, which requires source-level context or manual domain modeling.

3. The "Always Rewrite / Re-Engineer" Myth:

If a legacy component has ugly code, should we automatically re-engineer or rewrite it?

Counter-Argument: No! Apply the OORP pattern *"If It Ain't Broke, Don't Fix It"*. If an ugly component is rock-solid, has zero reported bugs, and zero incoming change requests, wrapping it behind a modern interface is far safer and more cost-effective than re-engineering or rewriting it.